P.Vishwa nandan(192473015)

TASK 6 – DESIGN CHALLENGE (13 MARKS)

Campus Event Registration Desktop Application

1. Aim

To design and implement a campus event registration desktop application using Java Swing and MySQL JDBC.

The application allows students to register for campus events and allows the cultural committee to view, filter, and export registration details.

The system must validate input, prevent duplicate registrations, and remain responsive while database operations are performed.

2. Problem Description

The college cultural committee needs a desktop application to manage student registrations for different campus events.

The application must provide the following features:

1. Students can enter their ID, name, email, and select an event.
2. Students can register for an event.
3. The committee can view all student registrations in a table.
4. The committee can filter registrations based on the selected event.
5. The committee can navigate through registration records using pagination.
6. The committee can export registration details into a CSV file.
7. Invalid input must be rejected with a clear error message.
8. A student must not be allowed to register twice for the same event.
9. Database operations must run in the background to prevent the GUI from freezing.

The application uses Java Swing for the interface and MySQL for storing registration details.

STAGE 1 – INTERPRET THE PROBLEM (2 MARKS)

3. Identify the Users

The application contains two main user groups.

3.1 Student

The student can:

- Enter personal details.
- Select a campus event.
- Register for an event.
- Receive a success or error message.

3.2 Cultural Committee

The committee can:

- View all student registrations.
- Filter registrations by event.
- Navigate through registration records.
- Export registration details into a CSV file.

3.3 Application Screens

The application contains two main screens.

Screen 1 – Student Registration Screen

Contains:

- Student ID field.
- Student name field.
- Email field.
- Event selection ComboBox.
- Register button.
- Clear button.
- Status message.

Screen 2 – Registration Management Screen

Contains:

- Event filter ComboBox.
- Registration JTable.
- Previous button.
- Next button.
- Export CSV button.
- Status message.

3.4 Data Requirements

The application stores two main types of data.

Events Table

Field	Description
event_id	Unique event identification number
event_name	Name of the event

active	Indicates whether the event is available

Registrations Table

Field	Description
registration_id	Unique registration number
student_id	Student identification number
student_name	Student's name
email	Student's email address
event_id	Selected event identification number
registered_at	Date and time of registration

3.5 Business Rules

1. Student ID must not be empty.
2. Student name must not be empty.
3. Email must have a valid format.
4. An event must be selected before registration.
5. A student can register only once for the same event.
6. Different students can register for the same event.
7. Registration details must be stored in MySQL.
8. Invalid registration requests must display meaningful error messages.
9. The committee must be able to filter registrations by event.
10. The committee must be able to export registration details.

3.6 Testable Requirements

Requirement	Expected Behaviour
Valid student registration	Registration is saved
Empty student ID	Error message appears
Empty student name	Error message appears
Invalid email	Registration is rejected
No event selected	Registration is rejected
Same student registers twice for one event	Duplicate registration is rejected

Different students select the same event	Both registrations are accepted
Select an event filter	Only matching registrations appear
Click Next	Next page appears
Click Prev	Previous page appears
Click Export	CSV file is generated
Load registration data	GUI remains responsive

STAGE 2 – PROPOSE THE SCREEN AND EVENT FLOW (2 MARKS)

4. Screen Design

4.1 Student Registration Screen

The student enters personal details and selects an event.

CAMPUS EVENT REGISTRATION

Student ID: [_____]

Student Name: [_____]

Email Address: [_____]

Select Event: [Select Event v]

[REGISTER] [CLEAR]

Status: Ready for registration.

4.2 Registration Management Screen

The committee can view and filter registration records.

REGISTRATION MANAGEMENT

Filter by Event: [All Events v]

| Reg ID | Student ID | Name | Email | Event |

| 1 | STU101 | Ravi | ... | Hackathon |

| 2 | STU102 | Arun | ... | Workshop |

| 3 | STU103 | Priya| ... | Cultural |

[PREV] [NEXT] [EXPORT CSV]

Status: Showing registrations.

4.3 Menu Bar Design

The application uses JMenuBar, JMenu, and JMenuItem.

File Menu:

- Export Registrations
- Exit

Navigate Menu:

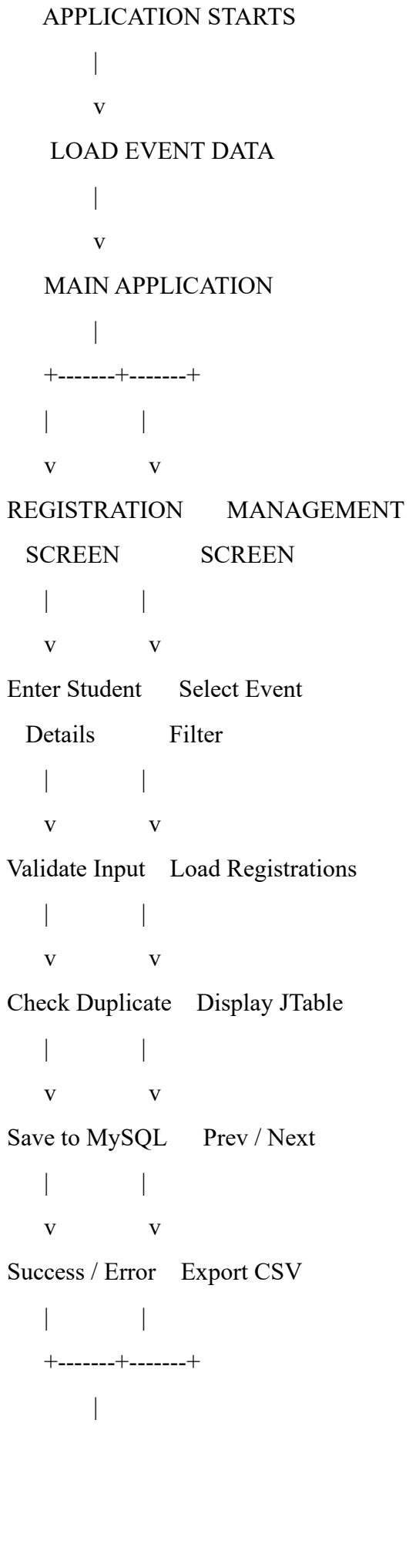
- Student Registration
- Manage Registrations

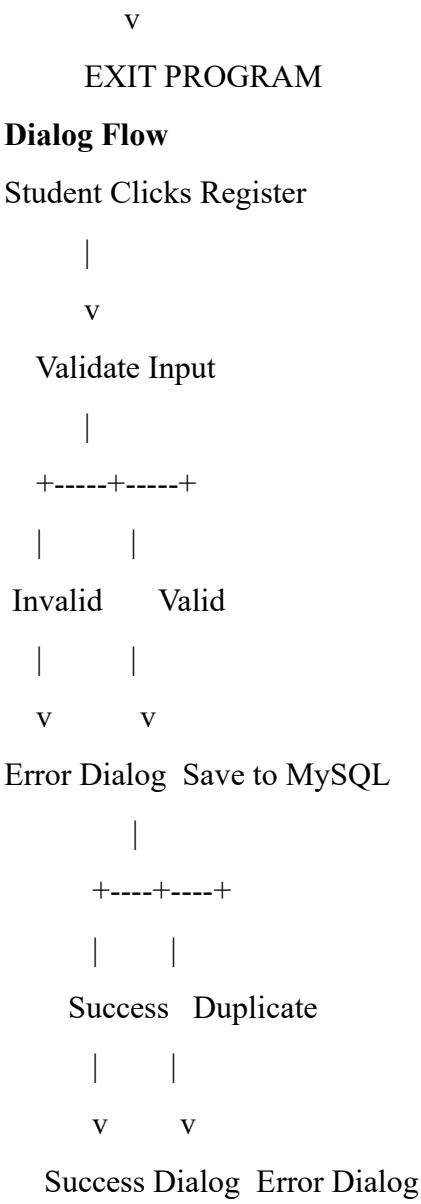
Help Menu:

- About Application

The menu bar allows the user to move between screens and access the application's main operations.

4.4 Screen Flow Diagram


APPLICATION STARTS
LOAD EVENT DATA
MAIN APPLICATION
REGISTRATION SCREEN
MANAGEMENT SCREEN
Enter Student Details
Select Event Filter
Validate Input
Load Registrations
Check Duplicate
Display JTable
Save to MySQL
Prev / Next
Success / Error
Export CSV



4.5 Dialog Flow

STAGE 3 – IDENTIFY THE CRITICAL DESIGN DECISIONS (2 MARKS)

5. Input Validation

Input validation is performed in the RegistrationService class.

The service checks:

- Student ID is not empty and follows the required format.
- Student name is not empty.
- Email contains a valid email format.
- An active event is selected.

If the input is invalid, a ValidationException is generated and displayed to the user.

5.2 Duplicate Registration Checking

Duplicate checking is handled using a MySQL UNIQUE constraint.

The database uses:

UNIQUE (student_id, event_id)

This prevents the same student from registering for the same event more than once.

For example:

Student ID	Event	Result
STU101	Hackathon	Accepted
STU101	Hackathon	Rejected
STU101	Workshop	Accepted
STU102	Hackathon	Accepted

The database constraint is important because two users may attempt to register simultaneously.

Checking duplicates only in Java is not sufficient to prevent concurrent duplicate registrations.

5.3 Database Access

The application uses JDBC to connect to MySQL.

Database operations are handled inside the RegistrationDAO class.

The DAO performs:

- Fetching event information.
- Inserting registration records.
- Fetching paginated registration records.
- Counting matching registrations.
- Exporting registration records into CSV.

5.4 Background Database Processing

Database operations may take time.

If they are executed directly on the Swing Event Dispatch Thread, the application window may freeze.

To prevent this, the application uses SwingWorker.

SwingWorker provides two important methods:

doInBackground()

Performs long-running operations in the background.

done()

Updates the GUI after the background operation finishes.

Example:

```
SwingWorker<Void, Void> worker =
   new SwingWorker<Void, Void>() {

   @Override
   protected Void doInBackground() throws Exception {

      // Database operation

      return null;
   }

   @Override
   protected void done() {

      // Update the interface
   }
};

worker.execute();
```

This keeps the application responsive while data is being loaded or saved.

STAGE 4 – DESIGN THE CLASSES AND HANDLERS (4 MARKS)

6. Layered Architecture

The application follows a three-layer architecture.

6.1 Presentation Layer

Class: RegistrationFrame

Responsibilities:

- Display the GUI.
- Collect student information.
- Handle button clicks.
- Display registration records.
- Show success and error messages.

- Handle filtering and export actions.

6.2 Business Logic Layer

Class: RegistrationService

Responsibilities:

- Validate student information.
- Check required fields.
- Apply registration rules.
- Call DAO methods.
- Handle registration-related business errors.

6.3 Data Access Layer

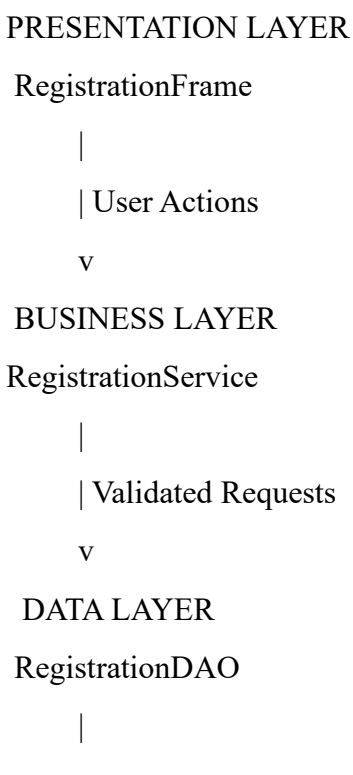
Class: RegistrationDAO

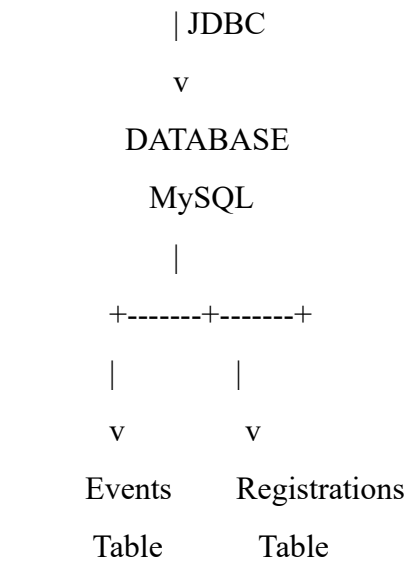
Responsibilities:

- Connect to MySQL.
- Insert registration records.
- Retrieve event information.
- Fetch paginated registration records.
- Export records to CSV.

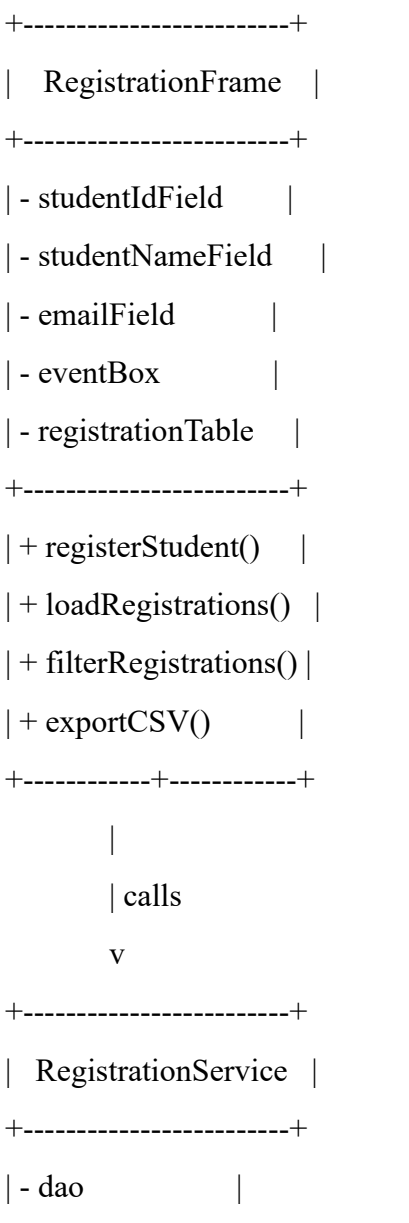
The Database class manages JDBC connections.

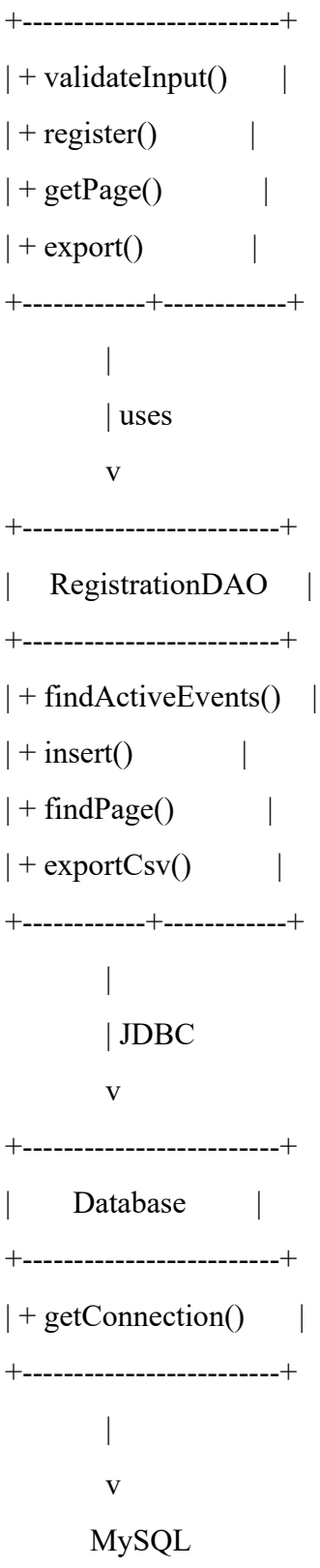
6.4 Architecture Diagram





6.5 Class Diagram





Supporting Classes:

Event

Registration

RegistrationPage

DuplicateRegistrationException

6.6 Pseudocode – Register Action

Aim: To validate student details and save a new event registration.

START

WHEN Register button is clicked:

Get student ID

Get student name

Get email address

Get selected event

Validate all input fields

IF any field is invalid THEN

Display validation error

ELSE

Start background worker

Check database registration constraints

Insert registration into MySQL

IF insertion succeeds THEN

Display success message

Clear input fields

Refresh registration table

ELSE IF duplicate registration occurs THEN

Display duplicate registration message

ELSE

Display database error

END IF

Finish background worker

STOP

6.7 Pseudocode – Filter Action

Aim: To display registrations belonging to the selected event.

START

WHEN Event filter changes:

Get selected event ID

Set current page to 0

Start SwingWorker

Fetch registration count

Fetch registrations using LIMIT and OFFSET

Update JTable with returned records

Update Previous and Next buttons

Display page status

STOP

STAGE 5 – IMPLEMENT AND DEMONSTRATE (3 MARKS)

7. Database Design

First, create the database and tables in MySQL Workbench.

7.1 SQL Program – campus_events.sql

Explanation:

1. The database campus_events stores the application information.
2. The events table stores available campus events.
3. The registrations table stores student registration details.
4. registration_id uniquely identifies each registration.
5. The foreign key connects registrations with events.
6. The UNIQUE constraint prevents duplicate registration by the same student for the same event.
7. The index improves event-based filtering and pagination.

7.2 Java Program 1 – Database.java

This class manages the MySQL database connection.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Database {
```

```
    private static final String URL =
        System.getenv().getOrDefault(
            "DB_URL",
            "jdbc:mysql://localhost:3306/campus_events"
        );

    private static final String USER =
        System.getenv().getOrDefault("DB_USER", "root");

    private static final String PASSWORD =
        System.getenv().getOrDefault("DB_PASSWORD", "");

    private Database() {
    }

    public static Connection getConnection()
            throws SQLException {

        return DriverManager.getConnection(
            URL, USER, PASSWORD
        );
    }
}
```

Explanation:

- URL specifies the MySQL database address.
- USER stores the database username.
- PASSWORD stores the database password.
- getConnection() creates and returns a database connection.
- Environment variables can be used to avoid writing database credentials directly in the program.

If environment variables are not configured, update the default username and password according to your MySQL setup.

7.3 Java Program 2 – Event.java

This class stores event information.

```
public class Event {

    private final int eventId;

    private final String eventName;

    public Event(int eventId, String eventName) {

        this.eventId = eventId;

        this.eventName = eventName;
    }

    public int getEventId() {

        return eventId;
    }

    public String getEventName() {

        return eventName;
    }

    @Override
    public String toString() {

        return eventName;
    }
}
```

Explanation:

- eventId stores the unique event number.

- eventName stores the event name.
- Getter methods provide access to event information.
- toString() displays the event name inside the JComboBox.

7.4 Java Program 3 – Registration.java

Explanation:

The Registration class stores all registration details.

It provides one constructor for creating new registrations and another constructor for storing records retrieved from MySQL.

7.5 Java Program 4 – RegistrationPage.java

This class stores the records returned for one page and the total matching record count.

```
import java.util.List;


public class RegistrationPage {


   private final List<Registration> registrations;


   private final int totalRecords;


   public RegistrationPage(
         List<Registration> registrations,
         int totalRecords) {


      this.registrations = registrations;


      this.totalRecords = totalRecords;
   }


   public List<Registration> getRegistrations() {


      return registrations;
   }
```

```
    public int getTotalRecords() {


        return totalRecords;
    }
}
```

Explanation:

The class stores:

- The registrations displayed on the current page.
- The total number of matching registrations.

This information is used to calculate the number of pages and control Prev and Next buttons.

7.6 Java Program 5 – DuplicateRegistrationException.java

This custom exception represents an attempt to register the same student for the same event more than once.

```
public class DuplicateRegistrationException extends Exception {


    public DuplicateRegistrationException(String message) {


        super(message);
    }
}
```

Explanation:

Instead of displaying a lengthy database error, the application uses this exception to show a meaningful duplicate-registration message.

7.7 Java Program 6 – RegistrationDAO.java

Explanation:

The RegistrationDAO class contains four main operations.

1. findActiveEvents()

Retrieves all active events from the events table.

2. insert()

Inserts a new registration into MySQL using PreparedStatement.

If the UNIQUE constraint detects a duplicate registration, the method throws a DuplicateRegistrationException.

3. findPage()

Retrieves a limited number of registration records using SQL LIMIT and OFFSET.

It also calculates the total number of matching records.

4. exportCsv()

Exports all matching registrations to a CSV file.

The records are written one at a time instead of loading the entire result into an ArrayList.

7.8 Java Program 7 – RegistrationService.java

Explanation:

The RegistrationService class validates all student information before sending the request to the DAO.

It checks the student ID, student name, email, and selected event.

It also provides methods for retrieving events, fetching registration pages, and exporting registration information.

The service separates business rules from the graphical user interface.

7.9 Java Program 8 – RegistrationFrame.java

7.10 Program Explanation

Step 1 – Starting the Application

The main() method starts the Swing application using:

```
SwingUtilities.invokeLater(() -> {
   RegistrationFrame frame = new RegistrationFrame();
   frame.setVisible(true);
});
```

This creates the GUI on the Swing Event Dispatch Thread.

Step 2 – Loading Events

The loadEvents() method retrieves available events from MySQL.

It uses SwingWorker so the database query does not block the interface.

The retrieved events are displayed in the registration and filter ComboBoxes.

Step 3 – Registering a Student

When the Register button is clicked, the application collects the entered student information.

The RegistrationService validates the input.

If the input is valid, the RegistrationDAO saves the information in MySQL.

Step 4 – Preventing Duplicate Registration

The database has a UNIQUE constraint on student ID and event ID.

If the same student tries to register for the same event again, MySQL rejects the insertion.

The DAO converts this database error into a DuplicateRegistrationException.

The GUI displays a meaningful error message.

Step 5 – Displaying Registration Records

The application retrieves only the required number of records.

The SQL query uses:

LIMIT 50 OFFSET 0

The JTable displays the returned records.

Step 6 – Filtering Records

The committee selects an event from the filter ComboBox.

The application loads only registrations belonging to that event.

Selecting All Events displays registrations from every event.

Step 7 – Pagination

The application displays 50 registrations per page.

- Prev loads the previous 50 records.
- Next loads the next 50 records.

The navigation buttons are enabled or disabled depending on the available records.

Step 8 – Exporting Registration Data

The Export CSV button opens a file chooser.

The selected event filter is applied to the export query.

The DAO retrieves matching records and writes them into a CSV file one at a time.

The export operation runs in SwingWorker, so the interface remains responsive.

8. Alternative Concept and Complexity Analysis

8.1 Alternative – Loading All Registrations at Once

An alternative approach is to retrieve all registration records from MySQL in one query and display them in the JTable.

Example:

```
SELECT
    r.registration_id,
    r.student_id,
    r.student_name,
    r.email,
    e.event_name,
```

```
    r.registered_at
FROM registrations r
JOIN events e ON r.event_id = e.event_id
ORDER BY r.registration_id;
```

The application loads all records into a collection and adds them to the JTable.

Advantages:

- All records are immediately available.
- No Prev or Next navigation is required.
- Filtering can be performed in memory.
- Suitable for small datasets.

Disadvantages:

- Transfers all records from the database.
- Requires more memory.
- Creates many Java objects.
- JTable updates can take longer.
- Large result sets can reduce application responsiveness.

8.2 Recommended Approach – LIMIT and OFFSET Pagination

The recommended approach loads only a limited number of records.

Example:

```
SELECT
    r.registration_id,
    r.student_id,
    r.student_name,
    r.email,
    e.event_name,
    r.registered_at
FROM registrations r
JOIN events e ON r.event_id = e.event_id
ORDER BY r.registration_id DESC
LIMIT 50 OFFSET 0;
```

Working:

1. The application requests the first 50 registrations.

2. MySQL returns only those records.
3. The JTable displays the records.
4. When Next is clicked, the application requests the next 50.
5. When Prev is clicked, it loads the previous page.

8.3 Time Complexity Analysis

Let:

- N = Total number of registration records.
- P = Number of records per page.
- O = Offset of the requested page.

Approach 1 – Load All Records

The database retrieves all N records.

Approximate time complexity:

O(N)

The application also processes all N returned records to create the table rows.

This requires more processing and data transfer.

Approach 2 – Pagination

The application retrieves only P records.

The Java-side processing per page is:

O(P)

The database uses LIMIT and OFFSET to select the requested page.

With a suitable index, the first pages can be retrieved efficiently. However, deep OFFSET queries may take longer because the database must skip preceding rows.

The separate COUNT query may also require work proportional to the number of matching records.

8.4 Space Complexity Analysis

Approach 1 – Load All Records

The application stores all registration records in memory.

Space complexity:

O(N)

For 100,000 registrations, all 100,000 records must be represented in Java objects and the table model.

This can consume substantial memory depending on the number and length of the text fields and Java object overhead.

Approach 2 – Pagination

The application stores only the records belonging to the current page.

Space complexity:

O(P)

For a page size of 50, only 50 registration records are held in the current table model.

This significantly reduces Java-side memory usage.

8.5 Comparison Table

Feature	**Load All Records**	**Pagination**
Records retrieved	All records	Only current page
Java-side processing	O(N)	O(P) per page
Java-side memory	O(N)	O(P)
Initial data transfer	Large	Small
Page navigation	Not required	Required
UI update workload	High for large datasets	Lower per page
Suitability	Small datasets	Large datasets
100,000 records	Resource intensive	More manageable

8.6 Effect on Application Responsiveness

Load All Records:

Loading 100,000 registrations may take longer due to database transfer, object creation, and JTable updates.

If performed on the Event Dispatch Thread, the application may freeze.

Even when performed in the background, building a table containing 100,000 rows may cause noticeable delays.

Pagination with SwingWorker:

The application retrieves only 50 records at a time.

The database operation runs inside doInBackground(), and the table is updated in done().

This allows the user to continue interacting with the application while records are being loaded.

Therefore, pagination with SwingWorker is more suitable for a large registration database.

8.7 Engineering Decision

The recommended design is SQL LIMIT and OFFSET pagination with SwingWorker.

The application uses a page size of 50 records.

This reduces the amount of data transferred for each request and reduces Java-side memory usage.

The index on event_id and registration_id supports event filtering and ordered retrieval.

For extremely large datasets where users frequently navigate to very deep pages, keyset pagination can be considered as a further improvement.

For CSV export, records are streamed directly from the ResultSet to the file instead of being stored in a large ArrayList.

9. Testing and Demonstration

The following test cases verify the application.

9.1 Normal Registration

Field	**Input**
Student ID	STU101
Student Name	Ravi Kumar
Email	ravi@example.com
Event	Hackathon

Expected Output:

Registration completed successfully!

The registration should appear in the MySQL registrations table.

9.2 Invalid Registration

Example input:

Student ID: STU102

Student Name: Arun

Email: invalid-email

Event: Hackathon

Expected Output:

Enter a valid email address.

The invalid registration must not be inserted into MySQL.

9.3 Duplicate Registration

Register STU101 for Hackathon again.

Expected Output:

This student is already registered for this event.

The database must reject the duplicate registration.

However, STU101 can register for a different event.

9.4 Event Filtering

Select:

Filter by Event: Hackathon

Expected Result:

Only students registered for Hackathon should appear in the JTable.

9.5 Pagination

Assume the database contains 125 registrations.

With a page size of 50:

Page	Records Displayed
Page 1	1–50
Page 2	51–100
Page 3	101–125

The Prev and Next buttons should work correctly.

9.7 Test Case Summary

Test Case	Input / Action	Expected Result
TC1	Valid registration	Registration saved
TC2	Empty student ID	Validation error
TC3	Empty student name	Validation error
TC4	Invalid email	Validation error
TC5	No event selected	Validation error
TC6	Duplicate student and event	Duplicate rejected
TC7	Same student selects another event	Registration accepted
TC8	Different students select same event	Both accepted
TC9	Filter by event	Matching records displayed
TC10	Click Next	Next page displayed
TC11	Click Prev	Previous page displayed

TC12	Export CSV	CSV file generated
TC13	Load large data	GUI remains responsive

10. Final Result

The Campus Event Registration Desktop Application was designed and implemented using Java Swing and MySQL JDBC.

The application separates the graphical interface, business logic, and database operations into different layers.

It supports student registration, input validation, event filtering, registration management, pagination, and CSV export.

A MySQL UNIQUE constraint prevents duplicate registrations for the same student and event.

SwingWorker is used to perform database operations in the background so that the application remains responsive.

The all-record loading approach was compared with SQL LIMIT and OFFSET pagination using time complexity, space complexity, and responsiveness.

Thus, the application demonstrates layered GUI design, database connectivity, duplicate prevention, background processing, and efficient management of large registration datasets.